МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт кибербезопасности и цифровых технологий (ИКБ)

КБ-2 «Информационно-аналитические системы кибербезопасности»

ОТЧЕТ О ВЫПОЛНЕНИИ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ №1

В РАМКАХ ДИСЦИПЛИНЫ «МЕТОДЫ АНАЛИЗА ДАННЫХ»

Выполнил:

Студент 4-ого курса

Учебной группы БИСО-02-22

Зубарев В.С.

Москва 2026

In [3]:
```
import pandas as pd
import numpy as np
import csv
import json
import bz2
import time
import datetime
from sklearn.model_selection import cross_val_score, KFold
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import roc_auc_score
import warnings
warnings.filterwarnings('ignore')

# Признаки, которые отсутствуют в тестовой выборке (информация после 5 минут)
target_columns = ['duration', 'radiant_win', 'tower_status_radiant',
                  'tower_status_dire', 'barracks_status_radiant', 'barracks_status_dire']

# Загрузка данных из CSV файла
features = pd.read_csv('./features.csv', index_col='match_id')


# Создаем копию для работы с градиентным бустингом
features_gb = features.copy()

# Удаляем признаки итогов матча (кроме radiant_win - это наша целевая переменная)
columns_to_drop = ['duration', 'tower_status_radiant', 'tower_status_dire',
                   'barracks_status_radiant', 'barracks_status_dire']
features_gb = features_gb.drop(columns=columns_to_drop, errors='ignore')

print(f'Размер после удаления: {features_gb.shape}')
print(f'Оставшиеся столбцы: {list(features_gb.columns)}')

# Проверка пропусков
missing_counts = features_gb.count()
total_rows = len(features_gb)
missing_info = pd.DataFrame({
    'Заполнено': missing_counts,
```

```
    'Пропусков': total_rows - missing_counts,
    '% пропусков': (total_rows - missing_counts) / total_rows * 100
})

# Показываем только столбцы с пропусками
missing_info = missing_info[missing_info['Пропусков'] > 0]
print('Столбцы с пропусками:')
print(missing_info)

# Сохраняем названия столбцов с пропусками для отчета
columns_with_missing = missing_info.index.tolist()
print(f'\nВсего столбцов с пропусками: {len(columns_with_missing)}')


# Заменяем пропуски на нули
features_gb = features_gb.fillna(0)
print('Пропуски заменены на нули')
print(f'Проверка: {features_gb.isnull().sum().sum()} пропусков осталось')

# Целевая переменная
target_column = 'radiant_win'
print(f'Целевая переменная: {target_column}')

# Разделяем признаки и целевую переменную
X = features_gb.drop(columns=[target_column])
y = features_gb[target_column]

print(f'Размер X: {X.shape}')
print(f'Размер y: {y.shape}')
print(f'Распределение классов:\n{y.value_counts()}')
print(f'Доля класса 1 (Radiant win): {y.mean():.4f}')

# Настройка кросс-валидации
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Тестируем разное количество деревьев
n_estimators_list = [10, 20, 30]
results_gb = []

print("Обучение градиентного бустинга...")
for n_est in n_estimators_list:
```

```
    start_time = datetime.datetime.now()

    gb_clf = GradientBoostingClassifier(
        n_estimators=n_est,
        random_state=42,
        max_depth=3  # Ограничиваем глубину для ускорения
    )

    scores = cross_val_score(gb_clf, X, y, cv=kf, scoring='roc_auc')

    end_time = datetime.datetime.now()
    elapsed = end_time - start_time

    results_gb.append({
        'n_estimators': n_est,
        'mean_auc': scores.mean(),
        'std_auc': scores.std(),
        'time': elapsed
    })

    print(f'n_estimators={n_est}: AUC-ROC = {scores.mean():.4f} (+/- {scores.std():.4f}), Время: {elapsed}')

results_gb_df = pd.DataFrame(results_gb)
results_gb_df

# Подготовка данных для логистической регрессии
features_lr = features.copy()
features_lr = features_lr.fillna(0)

# Удаляем признаки итогов матча
columns_to_drop = ['duration', 'tower_status_radiant', 'tower_status_dire',
                   'barracks_status_radiant', 'barracks_status_dire']
features_lr = features_lr.drop(columns=columns_to_drop, errors='ignore')

X_lr = features_lr.drop(columns=['radiant_win'])
y_lr = features_lr['radiant_win']

# Масштабирование признаков
scaler = StandardScaler()
X_lr_scaled = scaler.fit_transform(X_lr)
```

```python
# Подбор параметра регуляризации C
C_values = [0.01, 0.1, 1, 10, 100]
results_lr = []

print("Обучение логистической регрессии (все признаки)...")
for C in C_values:
    start_time = datetime.datetime.now()

    lr_clf = LogisticRegression(
        C=C,
        penalty='l2',
        random_state=42,
        max_iter=1000,
        solver='lbfgs'
    )

    scores = cross_val_score(lr_clf, X_lr_scaled, y_lr, cv=kf, scoring='roc_auc')

    end_time = datetime.datetime.now()
    elapsed = end_time - start_time

    results_lr.append({
        'C': C,
        'mean_auc': scores.mean(),
        'std_auc': scores.std(),
        'time': elapsed
    })

    print(f'C={C}: AUC-ROC = {scores.mean():.4f} (+/- {scores.std():.4f}), Время: {elapsed}')

results_lr_df = pd.DataFrame(results_lr)
best_C = results_lr_df.loc[results_lr_df['mean_auc'].idxmax(), 'C']
print(f'\nЛучший параметр C: {best_C}')
print(f'Лучшее качество AUC-ROC: {results_lr_df["mean_auc"].max():.4f}')

# Категориальные признаки
categorical_cols = ['lobby_type']
for i in range(1, 6):
    categorical_cols.append(f'r{i}_hero')
    categorical_cols.append(f'd{i}_hero')
```

```python
print(f'Категориальные признаки: {categorical_cols}')

# Удаляем категориальные признаки
X_lr_no_cat = X_lr.drop(columns=categorical_cols, errors='ignore')

# Масштабирование
scaler_no_cat = StandardScaler()
X_lr_no_cat_scaled = scaler_no_cat.fit_transform(X_lr_no_cat)

# Кросс-валидация с подбором C
results_lr_no_cat = []

print("Обучение логистической регрессии (без категорий)...")
for C in C_values:
    lr_clf = LogisticRegression(
        C=C,
        penalty='l2',
        random_state=42,
        max_iter=1000,
        solver='lbfgs'
    )

    scores = cross_val_score(lr_clf, X_lr_no_cat_scaled, y_lr, cv=kf, scoring='roc_auc')

    results_lr_no_cat.append({
        'C': C,
        'mean_auc': scores.mean(),
        'std_auc': scores.std()
    })

    print(f'C={C}: AUC-ROC = {scores.mean():.4f} (+/- {scores.std():.4f})')

results_lr_no_cat_df = pd.DataFrame(results_lr_no_cat)
best_C_no_cat = results_lr_no_cat_df.loc[results_lr_no_cat_df['mean_auc'].idxmax(), 'C']
print(f'\nЛучший параметр C (без категорий): {best_C_no_cat}')
print(f'Лучшее качество AUC-ROC: {results_lr_no_cat_df["mean_auc"].max():.4f}')

# Считаем количество уникальных героев
hero_cols = []
for i in range(1, 6):
    hero_cols.append(f'r{i}_hero')
```

```
    hero_cols.append(f'd{i}_hero')

all_heroes = []
for col in hero_cols:
    all_heroes.extend(features[col].dropna().unique())

n_heroes = len(set(all_heroes))
print(f'Количество различных идентификаторов героев: {n_heroes}')
print(f'Минимальный ID героя: {min(all_heroes)}')
print(f'Максимальный ID героя: {max(all_heroes)}')

# Создаем матрицу "мешок слов" для героев
# hero_id начинается с 1, поэтому делаем -1 для индексации
# Используем максимальный ID героя + 1 для размера матрицы

max_hero_id = int(features[hero_cols].max().max())
n_heroes = max_hero_id  # Количество признаков для героев

X_pick = np.zeros((features.shape[0], n_heroes))

print("Создание матрицы 'мешок слов' для героев...")
for i, match_id in enumerate(features.index):
    for p in range(5):
        # Radiant герои = +1
        r_hero = features.loc[match_id, f'r{p+1}_hero']
        if not pd.isna(r_hero) and r_hero > 0:
            X_pick[i, int(r_hero) - 1] = 1

        # Dire герои = -1
        d_hero = features.loc[match_id, f'd{p+1}_hero']
        if not pd.isna(d_hero) and d_hero > 0:
            X_pick[i, int(d_hero) - 1] = -1

print(f'Размер матрицы героев: {X_pick.shape}')
print(f'Сумма по строке (должна быть 0): {X_pick.sum(axis=1)[:5]}')

# Объединяем числовые признаки (без героев) с мешком слов
X_lr_bow = np.hstack([X_lr_no_cat_scaled, X_pick])
print(f'Итоговый размер признаков: {X_lr_bow.shape}')

# Кросс-валидация с подбором C
```

```python
results_lr_bow = []

print("Обучение логистической регрессии (с мешком слов)...")
for C in C_values:
    lr_clf = LogisticRegression(
        C=C,
        penalty='l2',
        random_state=42,
        max_iter=1000,
        solver='lbfgs'
    )

    scores = cross_val_score(lr_clf, X_lr_bow, y_lr, cv=kf, scoring='roc_auc')

    results_lr_bow.append({
        'C': C,
        'mean_auc': scores.mean(),
        'std_auc': scores.std()
    })

    print(f'C={C}: AUC-ROC = {scores.mean():.4f} (+/- {scores.std():.4f})')

results_lr_bow_df = pd.DataFrame(results_lr_bow)
best_C_bow = results_lr_bow_df.loc[results_lr_bow_df['mean_auc'].idxmax(), 'C']
print(f'\nЛучший параметр C (с мешком слов): {best_C_bow}')
print(f'Лучшее качество AUC-ROC: {results_lr_bow_df["mean_auc"].max():.4f}')

# Сводная таблица результатов
summary = pd.DataFrame({
    'Модель': ['Gradient Boosting (30 деревьев)',
               'Logistic Regression (все признаки)',
               'Logistic Regression (без категорий)',
               'Logistic Regression (с мешком слов)'],
    'Лучший AUC-ROC': [
        results_gb_df.loc[results_gb_df['n_estimators'] == 30, 'mean_auc'].values[0],
        results_lr_df['mean_auc'].max(),
        results_lr_no_cat_df['mean_auc'].max(),
        results_lr_bow_df['mean_auc'].max()
    ],
    'Лучший параметр': [
        f"n_estimators=30",
```

```
        f"C={best_C}",
        f"C={best_C_no_cat}",
        f"C={best_C_bow}"
    ]
})

print('=' * 70)
print('СВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ')
print('=' * 70)
print(summary.to_string(index=False))
print('=' * 70)

# Определяем лучшую модель
gb_auc_30 = results_gb_df.loc[results_gb_df['n_estimators'] == 30, 'mean_auc'].values[0]
lr_bow_auc = results_lr_bow_df['mean_auc'].max()

print(f'Лучшее качество Gradient Boosting (30 деревьев): {gb_auc_30:.4f}')
print(f'Лучшее качество Logistic Regression (с мешком слов): {lr_bow_auc:.4f}')

if gb_auc_30 >= lr_bow_auc:
    print('\nЛучшая модель: Gradient Boosting')
    best_model = GradientBoostingClassifier(
        n_estimators=30,
        random_state=42,
        max_depth=3
    )
    best_model.fit(X, y)
    use_gb = True
    best_auc = gb_auc_30
else:
    print('\nЛучшая модель: Logistic Regression с мешком слов')
    best_model = LogisticRegression(
        C=best_C_bow,
        penalty='l2',
        random_state=42,
        max_iter=1000,
        solver='lbfgs'
    )
    best_model.fit(X_lr_bow, y_lr)
    use_gb = False
    best_auc = lr_bow_auc
```

```
print(f'Лучшее AUC-ROC: {best_auc:.4f}')

# Загружаем тестовые данные
try:
    features_test = pd.read_csv('./features_test.csv', index_col='match_id')
    print(f'Размер тестовой выборки: {features_test.shape}')

    # Подготовка тестовых данных
    if use_gb:
        X_test = features_test.drop(columns=['duration', 'tower_status_radiant',
                                             'tower_status_dire', 'barracks_status_radiant',
                                             'barracks_status_dire'], errors='ignore')
        X_test = X_test.fillna(0)

        # Убеждаемся, что столбцы совпадают
        X_test = X_test[X.columns]

        predictions = best_model.predict_proba(X_test)[:, 1]
    else:
        # Для логистической регрессии с мешком слов
        X_test_no_cat = features_test.drop(columns=categorical_cols +
                                           ['duration', 'tower_status_radiant',
                                            'tower_status_dire', 'barracks_status_radiant',
                                            'barracks_status_dire'], errors='ignore')
        X_test_no_cat = X_test_no_cat.fillna(0)
        X_test_no_cat_scaled = scaler_no_cat.transform(X_test_no_cat)

        # Создаем мешок слов для теста
        X_test_pick = np.zeros((features_test.shape[0], n_heroes))
        for i, match_id in enumerate(features_test.index):
            for p in range(5):
                r_hero = features_test.loc[match_id, f'r{p+1}_hero']
                if not pd.isna(r_hero) and r_hero > 0:
                    X_test_pick[i, int(r_hero) - 1] = 1

                d_hero = features_test.loc[match_id, f'd{p+1}_hero']
                if not pd.isna(d_hero) and d_hero > 0:
                    X_test_pick[i, int(d_hero) - 1] = -1

        X_test_bow = np.hstack([X_test_no_cat_scaled, X_test_pick])
```

```
        predictions = best_model.predict_proba(X_test_bow)[:, 1]

    # Проверка предсказаний
    print(f'\n=== ПРОВЕРКА ПРЕДСКАЗАНИЙ ===')
    print(f'Минимальное предсказание: {predictions.min():.6f}')
    print(f'Максимальное предсказание: {predictions.max():.6f}')
    print(f'Среднее предсказание: {predictions.mean():.6f}')
    print(f'Все в диапазоне [0, 1]: {np.all((predictions >= 0) & (predictions <= 1))}')
    print(f'Не все одинаковые: {len(np.unique(predictions)) > 1}')

    # Сохраняем предсказания
    submission = pd.DataFrame({
        'match_id': features_test.index,
        'radiant_win': predictions
    })
    with open('submission.csv', 'w', newline='', encoding='utf-8') as f:
        writer = csv.writer(f)
        writer.writerow(['match_id', 'radiant_win'])
        for idx, row in submission.iterrows():
            writer.writerow([idx, row['radiant_win']])

    print('Предсказания сохранены в submission.csv')
    print(f'\nПредсказания сохранены в submission.csv')
    print(submission.head(10))

except FileNotFoundError:
    print('Файл features_test.csv не найден. Пропускаем этап предсказаний на тесте.')
    print('Убедитесь, что файл находится в той же директории.')
```

```
Размер после удаления: (97230, 103)
Оставшиеся столбцы: ['start_time', 'lobby_type', 'r1_hero', 'r1_level', 'r1_xp', 'r1_gold', 'r1_lh', 'r1_kills', 'r1_deaths',
'r1_items', 'r2_hero', 'r2_level', 'r2_xp', 'r2_gold', 'r2_lh', 'r2_kills', 'r2_deaths', 'r2_items', 'r3_hero', 'r3_level', 'r3
_xp', 'r3_gold', 'r3_lh', 'r3_kills', 'r3_deaths', 'r3_items', 'r4_hero', 'r4_level', 'r4_xp', 'r4_gold', 'r4_lh', 'r4_kills',
'r4_deaths', 'r4_items', 'r5_hero', 'r5_level', 'r5_xp', 'r5_gold', 'r5_lh', 'r5_kills', 'r5_deaths', 'r5_items', 'd1_hero', 'd
1_level', 'd1_xp', 'd1_gold', 'd1_lh', 'd1_kills', 'd1_deaths', 'd1_items', 'd2_hero', 'd2_level', 'd2_xp', 'd2_gold', 'd2_lh',
'd2_kills', 'd2_deaths', 'd2_items', 'd3_hero', 'd3_level', 'd3_xp', 'd3_gold', 'd3_lh', 'd3_kills', 'd3_deaths', 'd3_items',
'd4_hero', 'd4_level', 'd4_xp', 'd4_gold', 'd4_lh', 'd4_kills', 'd4_deaths', 'd4_items', 'd5_hero', 'd5_level', 'd5_xp', 'd5_go
ld', 'd5_lh', 'd5_kills', 'd5_deaths', 'd5_items', 'first_blood_time', 'first_blood_team', 'first_blood_player1', 'first_blood_
player2', 'radiant_bottle_time', 'radiant_courier_time', 'radiant_flying_courier_time', 'radiant_tpscroll_count', 'radiant_boot
s_count', 'radiant_ward_observer_count', 'radiant_ward_sentry_count', 'radiant_first_ward_time', 'dire_bottle_time', 'dire_cour
ier_time', 'dire_flying_courier_time', 'dire_tpscroll_count', 'dire_boots_count', 'dire_ward_observer_count', 'dire_ward_sentry
_count', 'dire_first_ward_time', 'radiant_win']
Столбцы с пропусками:
                              Заполнено  Пропусков  % пропусков
first_blood_time                  77677      19553    20.110048
first_blood_team                  77677      19553    20.110048
first_blood_player1               77677      19553    20.110048
first_blood_player2               53243      43987    45.240152
radiant_bottle_time               81539      15691    16.138023
radiant_courier_time              96538        692     0.711714
radiant_flying_courier_time       69751      27479    28.261853
radiant_first_ward_time           95394       1836     1.888306
dire_bottle_time                  81087      16143    16.602900
dire_courier_time                 96554        676     0.695259
dire_flying_courier_time          71132      26098    26.841510
dire_first_ward_time              95404       1826     1.878021

Всего столбцов с пропусками: 12
Пропуски заменены на нули
Проверка: 0 пропусков осталось
Целевая переменная: radiant_win
Размер X: (97230, 102)
Размер y: (97230,)
Распределение классов:
radiant_win
1    50414
0    46816
Name: count, dtype: int64
Доля класса 1 (Radiant win): 0.5185
Обучение градиентного бустинга...
```

```
n_estimators=10: AUC-ROC = 0.6649 (+/- 0.0010), Время: 0:00:25.079519
n_estimators=20: AUC-ROC = 0.6825 (+/- 0.0020), Время: 0:00:50.003879
n_estimators=30: AUC-ROC = 0.6900 (+/- 0.0021), Время: 0:01:14.749177
Обучение логистической регрессии (все признаки)...
C=0.01: AUC-ROC = 0.7165 (+/- 0.0012), Время: 0:00:00.564717
C=0.1: AUC-ROC = 0.7165 (+/- 0.0012), Время: 0:00:00.588792
C=1: AUC-ROC = 0.7165 (+/- 0.0012), Время: 0:00:00.601698
C=10: AUC-ROC = 0.7165 (+/- 0.0012), Время: 0:00:00.602478
C=100: AUC-ROC = 0.7165 (+/- 0.0012), Время: 0:00:00.601889

Лучший параметр C: 0.01
Лучшее качество AUC-ROC: 0.7165
Категориальные признаки: ['lobby_type', 'r1_hero', 'd1_hero', 'r2_hero', 'd2_hero', 'r3_hero', 'd3_hero', 'r4_hero', 'd4_hero',
'r5_hero', 'd5_hero']
Обучение логистической регрессии (без категорий)...
C=0.01: AUC-ROC = 0.7166 (+/- 0.0013)
C=0.1: AUC-ROC = 0.7165 (+/- 0.0013)
C=1: AUC-ROC = 0.7165 (+/- 0.0013)
C=10: AUC-ROC = 0.7165 (+/- 0.0013)
C=100: AUC-ROC = 0.7165 (+/- 0.0013)

Лучший параметр C (без категорий): 0.01
Лучшее качество AUC-ROC: 0.7166
Количество различных идентификаторов героев: 108
Минимальный ID героя: 1
Максимальный ID героя: 112
Создание матрицы 'мешок слов' для героев...
Размер матрицы героев: (97230, 112)
Сумма по строке (должна быть 0): [0. 0. 0. 0. 0.]
Итоговый размер признаков: (97230, 203)
Обучение логистической регрессии (с мешком слов)...
C=0.01: AUC-ROC = 0.7517 (+/- 0.0023)
C=0.1: AUC-ROC = 0.7519 (+/- 0.0024)
C=1: AUC-ROC = 0.7519 (+/- 0.0025)
C=10: AUC-ROC = 0.7519 (+/- 0.0025)
C=100: AUC-ROC = 0.7519 (+/- 0.0025)

Лучший параметр C (с мешком слов): 0.1
Лучшее качество AUC-ROC: 0.7519
======================================================================
СВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ
```

```
======================================================================
                             Модель  Лучший AUC-ROC Лучший параметр
    Gradient Boosting (30 деревьев)        0.690006 n_estimators=30
 Logistic Regression (все признаки)        0.716534          C=0.01
Logistic Regression (без категорий)        0.716553          C=0.01
Logistic Regression (с мешком слов)        0.751943           C=0.1
======================================================================
Лучшее качество Gradient Boosting (30 деревьев): 0.6900
Лучшее качество Logistic Regression (с мешком слов): 0.7519

Лучшая модель: Logistic Regression с мешком слов
Лучшее AUC-ROC: 0.7519
Размер тестовой выборки: (17177, 102)

=== ПРОВЕРКА ПРЕДСКАЗАНИЙ ===
Минимальное предсказание: 0.008428
Максимальное предсказание: 0.996374
Среднее предсказание: 0.517060
Все в диапазоне [0, 1]: True
Не все одинаковые: True
Предсказания сохранены в submission.csv

Предсказания сохранены в submission.csv
   match_id  radiant_win
0         6     0.822728
1         7     0.753668
2        10     0.187459
3        13     0.859613
4        16     0.239929
5        18     0.374287
6        19     0.530145
7        24     0.566178
8        33     0.213458
9        37     0.670821
```

In [ ]: